

UNIVERSIDAD DE EL SALVADOR

FACULTAD MULTIDISCIPLINARIA DE OCCIDENTE

DEPARTAMENTO DE INGENIERÍA Y ARQUITECTURA

Actividad: Guía de Ejercicios Prácticos en Ensamblador (NASM)

Modalidad: Grupos de trabajo ya asignados

Formato de Entrega: Un solo archivo de texto plano (.txt) por grupo, estrictamente un solo estudiante debe subir la solución. Porfavor acatar esta instrucción.

1. INDICACIONES GENERALES

1. **Trabajo en Equipo:** La presente guía debe ser resuelta exclusivamente por los grupos de trabajo previamente establecidos en el ciclo.
2. **Entorno de Desarrollo:** Todos los programas deben ser codificados utilizando **NASM (Netwide Assembler)**. Pueden utilizar el enlazador ld (para Linux) , según se haya definido en las sesiones de laboratorio.
3. **Formato de Entrega ÚNICO:** * Se deberá entregar **un solo archivo de texto con extensión .txt** a la plataforma virtual.
 - El nombre del archivo debe seguir el formato:
carnet1_carnet2_carnet3_Guia_NASM.txt
 - Dentro del archivo .txt, deben separar claramente cada ejercicio con un encabezado (Ejemplo: ### EJERCICIO 1 ###), seguido del código fuente del .asm completo y debidamente comentado.
4. **Buenas Prácticas:** Se evaluará el uso correcto de los registros, la gestión de la memoria (sección .data, .bss y .text) y los comentarios explicativos en las líneas clave del código.

2. GUÍA DE EJERCICIOS

Nivel Básico: Manipulación de Datos y Operaciones Aritméticas

1. Suma y Resta de Variables

Defina tres variables en la sección .data con valores numéricos de un byte (por ejemplo, \$A = 15\$, \$B = 7\$, \$C = 3\$). Diseñe un programa que calcule la operación \$Resultado = A + B - C\$ y almacene el valor final en un registro específico (o en una variable en .bss).

2. Multiplicación y División Básica

Solicite al usuario (o defina en memoria) dos números enteros de 8 bits. El programa debe multiplicar ambos números y, posteriormente, dividir el resultado entre 2. Asegúrese de manejar correctamente los registros de desbordamiento (AX / DX si aplica).

3. Intercambio de Valores (Sin usar memoria auxiliar)

Dadas dos variables en memoria, X e Y, con valores distintos, programe la lógica necesaria para intercambiar sus valores utilizando únicamente los registros del procesador (sin declarar una tercera variable de ayuda).

Nivel Intermedio: Estructuras de Control y Saltos Condicionales

4. Determinación de Par o Impar

Escriba un programa que tome un número entero almacenado en memoria y determine si es par o impar. El programa debe imprimir en pantalla el mensaje correspondiente ("El numero es Par" o "El numero es Impar") utilizando las llamadas al sistema (syscalls).

5. Mayor de Tres Números

Defina tres números enteros diferentes en la sección .data. Utilizando instrucciones de comparación (CMP) y saltos condicionales (JE, JG, JL, etc.), el programa debe identificar cuál de los tres números es el mayor y mostrarlo en la consola.

6. Contador Regresivo (Ciclos)

Diseñe un programa que realice un ciclo iterativo (LOOP o mediante saltos controlados) que vaya desde el número 9 hasta el 0. En cada iteración, el programa debe imprimir el número actual en pantalla, simulando una cuenta regresiva.

7. Multiplicación mediante Sumas Sucesivas

Para evitar el uso de la instrucción MUL, programe un algoritmo que reciba dos números (Multiplicando y Multiplicador) y realice la multiplicación ejecutando un ciclo que sume el "Multiplicando" tantas veces como indique el "Multiplicador".

Nivel Avanzado: Arreglos, Cadenas y Funciones

8. Sumatoria de un Arreglo de Datos

Defina un arreglo (*array*) de 5 números enteros de 16 bits en la sección `.data`. El programa debe recorrer el arreglo utilizando un registro como índice (o puntero) para sumar todos sus elementos y almacenar el total general en memoria.

9. Inversión de una Cadena de Texto

Dada una cadena de caracteres definida en memoria (por ejemplo: `db 'UES-FMO', 0`), diseñe un algoritmo que recorra la cadena y la imprima en pantalla en orden inverso (OMF-SEU). Puede apoyarse en el uso de la Pila (PUSH / POP) si lo considera conveniente.

10. Calculadora de Factorial (Estructura de Función)

Cree una subrutina o función llamada `calcular_factorial` que reciba un número entero pequeño (por ejemplo, entre 1 y 5) en el registro EBX / RBX y devuelva su factorial en el registro EAX / RAX. El flujo principal del programa debe llamar a la función e imprimir el resultado final.

3. CRITERIOS DE EVALUACIÓN (RÚBRICA)

- **Lógica y Correcto Funcionamiento (50%):** El código realiza exactamente lo que pide el enunciado sin errores de compilación o de ejecución.
- **Uso de Estructuras y Registros (20%):** Manejo adecuado de la arquitectura x86 (declaración correcta de secciones, uso apropiado de registros según su propósito).
- **Documentación y Comentarios (15%):** El código fuente dentro del `.txt` está debidamente comentado, explicando qué hace cada bloque o instrucción clave.
- **Formato de Entrega (15%):** Cumplimiento estricto de la entrega en un único archivo `.txt` con la estructura e identificación solicitada.